

Backend_Schema.md

SUMMARY

This document defines the backend architecture, data model, service boundaries, storage strategy, and communication patterns for Bloom.

PROJECT

Bloom

DOCUMENT

Backend Schema & Data Model

VERSION

1.0

STATUS

Backend Engineering Specification

PURPOSE

The objective is to create a scalable, modular, and maintainable backend capable of supporting millions of learners while powering AI-driven recommendations. This document is technology-agnostic. It describes logical architecture rather than implementation-specific databases or frameworks.

BACKEND PHILOSOPHY

Modular; Stateless; Event Driven; API First; AI Native.

HIGH-LEVEL BACKEND ARCHITECTURE

Client Applications (Browser Extension / Web App / Mobile) -> API Gateway -> Authentication, User Service, Learning Service -> Recommendation Service -> Skill Graph Service, Content Service, Creator Service -> Analytics Service, Notification Service -> Storage Layer.

SERVICE ARCHITECTURE

Authentication Service (Identity and access management); User Service (User profile management); Learning Service (Learning sessions and progress); Recommendation Service (AI recommendations); Skill Graph Service (Knowledge graph reasoning); Content Service (Educational resource metadata); Creator Service (Creator intelligence); Analytics Service (Event collection and metrics); Notification Service (User notifications); Search Service (Semantic and keyword search).

CORE DOMAIN MODEL

User: Learning Profile, Career Goal, Learning Sessions, Progress, Recommendations, Feedback. Career: Required Skills. Skill: Prerequisites, Resources, Creators, Assessments. Resource: Creator, Platform, Skills, Metadata.

PRIMARY ENTITIES

User, Learning Profile, Career, Skill, Resource, Creator, Learning Session, Recommendation, Feedback with listed fields for each entity.

RELATIONSHIPS

User 1:N Learning Session, 1:1 Learning Profile, 1:N Recommendation, 1:N Feedback. Career 1:N Skills. Skill N:N Resources, N:N Creators, N:N Prerequisite Skills. Creator 1:N Resources.

STORAGE STRATEGY

User Data (Relational), Learning Profiles (Relational), Recommendations (Relational), Learning Sessions (Relational), Analytics Events (Time-Series/Event Store), Skill Graph (Graph Database), Career Graph (Graph Database), Content Metadata (Document Store), Search Index (Search Engine), Cache (In-Memory Cache).

EVENT MODEL

User Registered -> Profile Created -> Career Selected -> Recommendation Generated -> Learning Started -> Learning Completed -> Feedback Submitted -> Profile Updated. Each event may trigger downstream services asynchronously.

RECOMMENDATION PIPELINE

Learning Request -> Read Learning Profile -> Skill Gap Analysis -> Retrieve Candidate Resources -> Rank Candidates -> Generate Explanation -> Store Recommendation -> Return Recommendation.

ANALYTICS SCHEMA

Examples: User Login, Resource Viewed, Resource Completed, Recommendation Accepted, Recommendation Skipped, Search Performed, Learning Session Started, Learning Session Completed. Each event contains: Event ID, User ID, Event Type, Timestamp, Metadata.

SEARCH SCHEMA

Indexes: Skills, Careers, Creators, Resources, Projects, Technologies. Supports: Keyword search, Semantic search, Auto-complete, Filtering, Ranking.

API COMMUNICATION

Synchronous APIs for request-response workflows; Asynchronous events for long-running operations; Idempotent operations where possible; Versioned interfaces; Structured error handling.

CACHING STRATEGY

Frequently accessed data cached: Learning profiles, Recommendations, Skill graph fragments, Popular resources, Creator metadata. Cache invalidation occurs when underlying data changes.

BACKGROUND PROCESSING

Asynchronous tasks include Recommendation generation, Content indexing, Skill graph updates, Analytics aggregation, Notification delivery, Resource quality evaluation.

SECURITY MODEL

Protected resources require authentication. Authorization is role-based. Possible roles: Learner, Administrator, Organization Manager, Enterprise Administrator, System Service. All sensitive operations are audited.

DATA LIFECYCLE

Create -> Validate -> Store -> Use -> Update -> Archive -> Delete. Data retention policies should comply with applicable privacy regulations.

SCALABILITY STRATEGY

Horizontal service scaling, Independent database scaling, Read replicas, Distributed caching, Asynchronous processing, Load balancing. No service should require full-system downtime for scaling.

FAILURE RECOVERY

Retry transient failures; Use cached responses where appropriate; Queue asynchronous work; Degrade gracefully; Log failures for analysis. Critical learner data must remain durable.

MONITORING & OBSERVABILITY

Metrics: Request count, Latency, Error rate, Throughput. Logs: Structured logs, Audit logs, Error logs. Tracing: End-to-end request tracing, Cross-service dependency tracking.

FUTURE EXTENSIONS

Mobile applications, Offline synchronization, Multi-region deployments, AI model versioning, Plugin ecosystem, Public APIs, Institutional tenants, Enterprise integrations. These additions should require minimal architectural changes.

BACKEND SUMMARY

Bloom's backend is designed as a modular, service-oriented architecture centered around learner intelligence rather than content delivery. It separates user management, AI reasoning, knowledge graphs, content management, analytics, and notifications into independently scalable services, with domain-appropriate storage models for flexibility and long-term scalability.